

AgroCafé — Guía de Despliegue de Base de Datos

Manual Técnico de Instalación, Arquitectura y Operación en PostgreSQL 14+

PROYECTO

AgroCafé (Integrador)

MOTOR OBJETIVO

PostgreSQL 14+

MÓDULO

Base de Datos y Disparadores

Propósito del documento: Esta guía contiene las instrucciones paso a paso para la correcta ejecución del script de base de datos de **AgroCafé**, la explicación funcional de sus componentes automatizados (Triggers, Vistas, Auditoría) y las pautas para integrar la base de datos con el backend en PHP y la API en Python/FastAPI.

1. Requisitos Previos y Corrección Obligatoria

Antes de ejecutar el script en tu entorno local o en Docker, asegúrate de haber aplicado la corrección de sintaxis obligatoria en el archivo SQL:

Corrección en la función de auditoría:

En la definición de `fn_registrar_bitacora()`, la línea de excepción debe decir `EXCEPTION WHEN OTHERS THEN` (corrigiendo el error de tipeo que tenía dos "N" al final: `THENN`).

```
-- Corrección requerida en el script SQL
CREATE OR REPLACE FUNCTION fn_registrar_bitacora()
RETURNS TRIGGER AS $$
DECLARE
    v_id_usuario INTEGER;
BEGIN
    BEGIN
        v_id_usuario := current_setting('app.usuario_actual', true)::INTEGER;
    EXCEPTION WHEN OTHERS THEN -- Correcto: THEN (sin la 'N' extra)
        v_id_usuario := NULL;
    END;
    ...
END;
$$ LANGUAGE plpgsql;
```

2. Pasos de Ejecución (Paso a Paso)

Paso 1: Preparación del Entorno (Evitar el error "already exists")

Si intentaste correr el script previamente y falló a mitad de camino, las tablas parciales impedirán reejecutarlo. Ejecuta el siguiente comando para limpiar el esquema `public` o crea una base de datos totalmente nueva:

```
-- Opción A: Limpiar el esquema actual (¡Borra tablas existentes!)
DROP SCHEMA public CASCADE;
CREATE SCHEMA public;

-- Opción B: Crear una base de datos limpia desde psql / pgAdmin
CREATE DATABASE agrocafe_db;
\c agrocafe_db
```

Paso 2: Cargar y Ejecutar el Script DDL

Abre el archivo SQL corregido en **pgAdmin (Query Tool)**, **DBeaver** o desde la terminal interactiva con `psql`:

```
psql -U postgres -d agrocafe_db -f agrocafe_schema.sql
```

Si la ejecución es exitosa, el motor confirmará la creación de las 14 tablas, 13 índices, 4 funciones, 6 triggers y 4 vistas.

Paso 3: Crear el Usuario de Aplicación (Seguridad)

Para evitar que el backend en PHP se conecte como superusuario (postgres), ejecuta el apéndice de seguridad:

```
CREATE ROLE agrocafe_app WITH LOGIN PASSWORD 'TuContrasenaSegura123!';
GRANT CONNECT ON DATABASE agrocafe_db TO agrocafe_app;
GRANT USAGE ON SCHEMA public TO agrocafe_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO agrocafe_app;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO agrocafe_app;
```

3. Explicación de la Arquitectura de Datos

El esquema está diseñado en **Tercera Forma Normal (3NF)** y dividido en 5 áreas funcionales principales:

Área Funcional	Tablas Principales	Descripción del F
1. Catálogos	roles, usuarios, parcelas, temporadas, tipos_cafe, almacenes, catalogo_tipos_movimiento	Contienen los dato requeridos para po formularios y relac operaciones. Debe información en est antes de registrar l
2. Producción	lotes_cosecha, datos_agronomicos	Representa las cos recolectadas. Cada en estado 'pendiente_anal que es procesado de IA.
3. Motor IA	analisis_ejecuciones, clasificaciones_ia	Almacena cada co modelo K-Means re FastAPI y la asigna clúster correspondi cada lote.
4. Inventario	inventario_existencias, movimientos_inventario, alertas_inventario	Control de existenci por almacén, histor movimientos (entra traslados, mermas notificaciones auto
5. Auditoría	bitacora_operaciones	Registro centraliza cambios (INSERT/ DELETE) guardand

Área Funcional	Tablas Principales	Descripción del F
		completo del regist JSONB.

4. Automatizaciones Integradas (Triggers y Vistas)

Triggers Automáticos

- **Sincronización de Inventario (trg_procesar_movimiento):** El backend PHP NO debe modificar inventario_existencias manualmente. Solo inserta en movimientos_inventario y la base de datos actualizará/creará las existencias de forma transparente.
- **Clasificación de Lote (trg_marcar_lote_clasificado):** Al insertar el resultado del K-Means en clasificaciones_ia, el estado del lote en lotes_cosecha pasa automáticamente a 'clasificado'.
- **Alertas de Stock Bajo (trg_generar_alerta_stock):** Si la cantidad disponible de un lote cae por debajo o igual a su stock_minimo, se crea un registro en alertas_inventario.
- **Actualización de Tiempos (updated_at):** Mantiene la fecha de modificación al día en tablas clave.

Vistas para Dashboard e Integración

- vista_lotes_pendientes_analisis: Utilizada por PHP para consultar los lotes sin clasificar y enviarlos mediante HTTP/cURL a la API de FastAPI.
- vista_inventario_actual: Alimenta el tablero principal con la existencia, almacén y la categoría asignada por la IA.
- vista_alertas_activas: Muestra los lotes que requieren reabastecimiento o atención urgente.
- vista_distribucion_clusters: Proporciona la distribución requerida para las gráficas de rendimiento del Dashboard.

5. Guía de Integración para Backend PHP

Orden Estricto de Inserción de Datos:
Para poblar la base de datos sin violar las llaves foráneas (Foreign Keys), sigue este orden:

1. Insertar roles → Insertar usuarios.
2. Insertar parcelas, temporadas, tipos_cafe, almacenes.
3. Insertar lotes_cosecha y sus correspondientes datos_agronomicos.

Asociar Auditoría al Usuario Actual (Sesión PHP)

Para que la tabla bitacora_operaciones guarde el ID del usuario que realizó la acción desde PHP, la aplicación debe ejecutar esta sentencia dentro de la misma transacción antes de hacer cambios:

```
-- En PHP mediante PDO:  
$pdo->exec("SET LOCAL app.usuario_actual = '{$idUserioSesion}'");
```